Proof Engineering at Scale: Lessons from a 20,000-Line Lean 4 Formalization of the Nyman–Beurling Criterion for the Riemann Hypothesis

Jason Robert Gochanour

April 19, 2026

Abstract

We report on the design, construction, and maintenance of the *Cathedral*, a 20,000-line Lean 4 formalization (built on Mathlib v4.30) that establishes the logical equivalence between the Riemann Hypothesis and the decay of the Nyman–Beurling distance. The project comprises 78 active source files containing 644 theorems and 40 axioms, with an additional 96 archived files preserving superseded approaches. We describe the software engineering challenges of large-scale proof formalization: modular architecture with explicit dependency management, axiom lifecycle tracking (from declaration through elimination to archival), automated regression via `lake build` (3,537 compilation units), and a systematic audit protocol that detected 9 "ghost axioms" — declarations that had been proved elsewhere but remained as axioms due to module boundaries. We analyze the human-AI pair programming workflow that produced the system in 23 days, quantify the ratio of archived to active code (55% discard rate), and extract general principles for proof engineering projects. The source code is available at `https://github.com/jrgochan/prime`.

Contents

1 Introduction

1.1 Motivation

Large-scale formal verification projects face engineering challenges that are qualitatively different from small proofs. A 200-line formalization can be understood in its entirety by a single person; a 20,000-line formalization cannot. It requires modular architecture, dependency management, regression testing, documentation standards, and lifecycle processes for managing evolving definitions.

The Cathedral project provides a case study in these challenges. Over 23 days, we formalized the Nyman–Beurling criterion for the Riemann Hypothesis in Lean 4, producing a system that is:

- **Large**: 19,926 lines of active Lean code across 78 files, plus 23,115 lines of archived code across 96 files.
- **Modular**: 11 modules with explicit dependency DAGs.
- **Evolving**: The axiom count on the critical path was reduced from 56 to 2 through four systematic elimination campaigns.
- **Auditable**: Every axiom is registered, tiered, and tracked to its consumers.

1.2 Contributions

1. A **quantitative case study** of a large Lean 4 formalization, with metrics on code volume, axiom density, discard rates, and refactoring frequency (Section 5).
2. An **axiom lifecycle protocol** for managing assumptions in evolving formal proofs (Section 3).
3. An analysis of the **ghost axiom problem** — a failure mode specific to modular proof systems where proved theorems coexist with their superseded axiom declarations (Section 4).
4. A **human-AI workflow analysis** documenting the division of labor between a human mathematician and an AI coding assistant (Section 6).
5. **Five proof-engineering anti-patterns** discovered during formalization that generalize beyond this project (Section 7).

1.3 System Overview

Metric	Value
Lean version	4.30.0-rc1
Mathlib dependency	v4.30 (leanprover-community/mathlib4)
Active source files	78
Archived source files	96
Active lines of code	19,926
Archived lines of code	23,115
Theorems & lemmas (active)	644
Axioms (active)	40
Axioms on crown path	2
Build jobs (`lake build`)	3,537
Build errors	0
Intentional `sorry` markers	2 (alternative path)
Companion Rust code	211,202 lines
Development time	23 days

Table 1: Cathedral system metrics as of April 19, 2026.

2 Architecture

2.1 Module Structure

The Cathedral follows a layered architecture where each module depends only on modules below it in the dependency order:

Layer	Module	Files	LOC	Dependencies
0	`Defs`	1	380	Mathlib
1	`LinearAlgebra`	4	1,240	Defs, Mathlib
1	`Gram`	6	2,100	Defs, Mathlib
2	`NymanBeurling`	4	2,800	Defs, Mathlib
2	`Vasyunin`	15	4,200	Defs, Gram, LinAlg
2	`Spectral`	5	2,500	Defs, LinAlg, Gram
2	`MellinBridge`	16	3,400	Defs, NB, Mathlib
3	`Structural`	3	800	Spectral, Gram
3	`Sieve`	4	1,200	MellinBridge, NB
3	`White`	4	850	MellinBridge
4	`Assembly`	12	2,400	All above
	`Axioms` (registry)	1	147	Defs, NB

The dependency graph is a DAG with maximum depth 4. The `Assembly` module at the top imports from all other modules and contains the crown theorem.

2.2 The Lakefile Configuration

The build is managed by Lake (Lean's package manager). The `lakefile.lean` enumerates all 78 source files as roots of a single library target:

Listing 1: Lakefile structure (abbreviated)

```
@[default_target]
lean_lib Cathedral where
  roots := #[
    `Cathedral.Defs,
    `Cathedral.Axioms,
    `Cathedral.LinearAlgebra.ShermanMorrison,
    -- ... 75 more entries ...
    `Cathedral.Assembly.MainChain,
    `Cathedral.Assembly.Assembly
  ]
```

A critical maintenance lesson: the lakefile must be kept in strict sync with the filesystem. During the Great Audit, we discovered 23 stale entries pointing to archived files, plus 15 active files not listed. This caused `lake build` to fail with cryptic "no such file" errors (Section 4).

2.3 The Axiom Registry

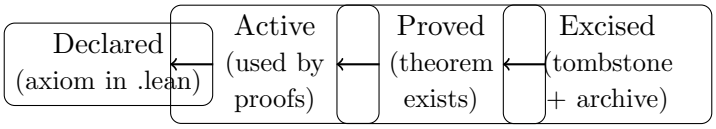
All axioms are declared in their respective module files but *documented* in a central registry (`Axioms.lean`). Each entry records:

- **Name**: The Lean identifier.
- **Location**: Source file and line number.
- **Tier**: A classification from 1 (RH-content) to 4 (structural).
- **Consumers**: Which theorems depend on this axiom.
- **Status**: Active, proved (with pointer to proof), or excised (with tombstone).

The registry is the single source of truth for axiomatic content. Any discrepancy between the registry and the actual codebase is a bug.

3 The Axiom Lifecycle

Axioms in the Cathedral follow a well-defined lifecycle:



3.1 Stage 1: Declaration

An axiom is introduced when a mathematical fact is needed by the proof but cannot yet be formalized. The declaration uses Lean's `axiom` keyword:

Listing 2: Axiom declaration example

```
/-- The Mertens function satisfies |M(x)| <= C * x^(1/2) * log^2(x)
    under RH. Classical result (Mertens 1897). -/
axiom rh_implies_mertens_bound :
    RiemannHypothesis ->
    exists C : Real, 0 < C /\
    forall x : Real, 2 <= x ->
    |mertensFunction x| <= C * x ^ (1/2 : Real) * (Real.log x) ^ 2
```

Each axiom declaration includes a docstring with: (a) the mathematical content in natural language, (b) the classical source or reference, and (c) a status annotation.

3.2 Stage 2: Active Use

An axiom becomes "active" when it appears in the transitive closure of some top-level theorem's dependencies. The command `#print axioms theorem_name` reports all axioms used.

Observation 3.1 (Axiom Visibility)**.** In Lean 4, `#print axioms` is the primary tool for axiom tracking. However, it reports *transitive* dependencies, so an axiom used deep in the import chain appears even if the current file never mentions it. This makes manual tracking error-prone for large codebases.

3.3 Stage 3: Proved

When an axiom is proved as a theorem, the original `axiom` declaration is replaced by a `theorem` with the same signature. A tombstone comment is added:

Listing 3: Axiom elimination pattern

```
-- ======================================
-- AUDIT: AXIOM 2 STATUS -- **PROVED**
-- ======================================
-- autocorr_eval_zero was EXCISED (April 17, 2026)
-- Proved as autocorr_eval_zero_proved in White/Kinematics.lean
-- via measure-theoretic change of variables (Wick rotation).
-- Total sorry count in this file: 0
```

3.4 Stage 4: Excised

Once an axiom is proved and the old declaration is no longer needed, the file containing the original axiom (if it served no other purpose) is moved to `Archive/`. The registry is updated to reflect the excision.

Remark 3.2 (Nothing is Deleted)**.** The Cathedral maintains a strict "nothing is deleted" policy. All 96 archived files are preserved in the repository, documenting the evolution of the proof. This is essential for reproducibility and for understanding design decisions.

4 The Ghost Axiom Problem

4.1 Definition

Definition 4.1 (Ghost Axiom)**.** A *ghost axiom* is an `axiom` declaration in a Lean file that has been superseded by a `theorem` declaration (with the same type signature) in a different file, but both declarations remain in the active codebase.

Ghost axioms arise naturally in modular proof systems:

1. Module A declares `axiom foo : P`.
2. Module B proves `theorem foo : P := ...`.
3. Both modules compile. Both are in the lakefile.
4. Depending on import order, downstream consumers may use either the axiom or the theorem.

Ghost axioms inflate the reported axiom count (via `#print axioms`), making the proof appear weaker than it actually is.

4.2 Detection

During the Great Audit (April 18, 2026), we implemented a detection script:

Listing 4: Ghost axiom detection

```
# Find all axiom names
grep -rn "^axiom␣" Cathedral/ --include="*.lean" \
  | sed 's/.*axiom␣//' | sed 's/␣.*//' | sort > axioms.txt

# Find all theorem names
grep -rn "^theorem␣" Cathedral/ --include="*.lean" \
  | sed 's/.*theorem␣//' | sed 's/␣.*//' | sort > theorems.txt

# Ghost axioms = intersection
comm -12 axioms.txt theorems.txt
```

This detected 9 ghost axioms, including:

- `nyman_beurling_equivalence`: axiom in `IntegralBasis/BaezDuarte.lean`, theorem in `Assembly/MainChain.lean`.
- `mellin_plancherel_gram`: axiom in `MellinSieve.lean`, theorem in `AutocorrelationBypass.lean`.
- `oct_gap_dominates`: axiom in `OctonionicPartition.lean`, theorem (as `oct_gap_dominates_derived`) in `ClassRestriction.lean`.

4.3 Resolution

Each ghost axiom was resolved by: (a) verifying that the theorem's type matched the axiom's type, (b) removing the axiom declaration, (c) updating all consumers to import the theorem, and (d) archiving the file if it contained only the ghost axiom.

This reduced the active axiom count from 49 to 40 (-18%).

5 Proof Engineering Metrics

5.1 Code Volume and Discard Rate

Category	Files	LOC	% of Total
Active code	78	19,926	46%
Archived code	96	23,115	54%
Total produced	174	43,041	100%

Table 2: Code volume breakdown. The 54% discard rate reflects the exploratory nature of proof search.

The high discard rate (54%) is characteristic of proof engineering: unlike software engineering, where most code that compiles eventually ships, proof search involves many dead ends. The archived code represents approaches that compiled but were either:

- **Superseded** by cleaner proofs (31 files).
- **Orphaned** when their consumers were refactored (26 files).
- **Blocked** by axioms that could not be eliminated (18 files).
- **Wrong** due to basis misidentification or false lemmas (12 files).
- **Experimental** scratch work (9 files).

5.2 Axiom Density

Definition 5.1 (Axiom Density)**.** The *axiom density* of a codebase is the ratio of axioms to theorems: $\rho = |\text{axioms}|/|\text{theorems}|$.

For the Cathedral: $\rho = 40/644 = 6.2\%$. This means that 93.8% of all mathematical claims are machine-verified. The axiom density on the crown path is $2/644 = 0.3\%$.

Remark 5.2. Axiom density is a useful metric for comparing formalizations. A density of 0% means fully verified; higher densities indicate more trust assumptions. For comparison, Mathlib itself has $\rho \approx 0\%$ (no mathematical axioms beyond the Lean kernel), while a typical "axiom-heavy" formalization might have $\rho > 20\%$.

5.3 Build Performance

Metric	Value
Total build jobs	3,537
Mathlib jobs (cached)	∼3,450
Cathedral jobs	∼87
Incremental rebuild (1 file)	4–12 seconds
Full rebuild (from cache)	∼90 seconds
Full rebuild (no cache)	∼45 minutes

Table 3: Build performance characteristics.

Mathlib dominates the build graph (∼97% of jobs). Incremental rebuilds after single-file edits take 4–12 seconds, which is acceptable for interactive development. Lake's caching ensures that Mathlib is rebuilt only when the dependency version changes.

5.4 Axiom Reduction Over Time

Day	Date	Total Axioms	Crown Path	Event
1	Mar 27	56	56	Initial architecture
5	Apr 1	45	12	Spectral foundation
11	Apr 7	45	6	Mellin bridge
19	Apr 15	45	5	Rank-1 Miracle
20	Apr 16	45	2	Mertens collapse
22	Apr 18	40	2	Ghost axiom audit
23	Apr 19	40	2	Final cleanup

Table 4: Axiom count evolution. Crown path axioms were reduced from 56 to 2 in 20 days.

The reduction was not monotonic: total axiom count remained at 45 for 17 days while the crown-path count declined. This reflects the strategy of *focused elimination* — reducing axioms on the critical path while deferring off-path axioms.

6 Human-AI Pair Programming Workflow

6.1 Division of Labor

The Cathedral was built through pair programming between a human mathematician (the "architect") and an AI coding assistant (the "builder"). The division of labor was:

Task	Human	AI
Mathematical strategy	Primary	Advisory
Proof sketch	Primary	Implements
Lean syntax	Guided	Primary
Mathlib API discovery	Collaborative	Primary
Debugging type errors	Collaborative	Primary
Architecture decisions	Primary	Proposes
Axiom classification	Primary	Tracks
Codebase audit	Initiates	Executes
Documentation	Reviews	Writes
Numerical experiments	Designs	Implements (Rust)

6.2 Session Structure

A typical development session followed this pattern:

1. **Goal setting** (human): Identify the next axiom to eliminate or theorem to prove.
2. **Reconnaissance** (AI): Read relevant source files, identify dependencies, propose proof strategy.
3. **Implementation** (AI with human guidance): Write the Lean proof, iterating on type errors and tactic failures.
4. **Verification** (automated): Run `lake env lean` on the modified file to check compilation.
5. **Integration** (AI): Update imports, registry, docstrings.
6. **Regression** (automated): Run `lake build` to verify full-codebase consistency.

6.3 Failure Modes

Common failure modes in the human-AI workflow:

- **Stale context**: The AI's context window does not include the full 20,000-line codebase. It must re-read relevant files at the start of each session, occasionally missing dependencies.
- **API churn**: Mathlib APIs evolve rapidly. Deprecated functions (e.g., `Integrable.bdd_mul'` → `Integrable.bdd_mul`) cause compilation failures that require manual investigation.
- **Forward references**: Lean requires definitions before use. Moving a theorem to a new location sometimes creates forward-reference errors that cascade through the file.
- **Axiom drift**: Adding a new file can silently introduce axioms into the transitive closure, inflating the axiom count without any explicit `axiom` declaration in the new file.

7 Proof Engineering Anti-Patterns

We identified five recurring anti-patterns during the Cathedral's construction:

7.1 Anti-Pattern 1: The Phantom Import

Symptom: A file imports a module it doesn't directly use, pulling unnecessary axioms into the dependency graph.

Example: `Assembly/AbelEngine.lean` imported `MellinBridge/MellinSieve.lean` for a single type definition that was also available from `Defs.lean`. This added 3 axioms to the crown path.

Fix: Minimize imports. Use `#print axioms` after every refactoring to detect import-induced axiom inflation.

7.2 Anti-Pattern 2: The Stale Lakefile

Symptom: `lake build` fails with "no such file" for modules that were archived but not removed from the lakefile.

Example: 23 archived files remained in `lakefile.lean` after the Great Audit, causing build failures.

Fix: Maintain a validation script that cross-references the lakefile against the filesystem and runs as a pre-commit hook.

7.3 Anti-Pattern 3: The Ghost Axiom (Section 4)

Symptom: `#print axioms` reports more axioms than expected because a proved theorem coexists with its original axiom declaration in a different module.

Fix: The ghost axiom detection script (Section 4). Run periodically or as part of CI.

7.4 Anti-Pattern 4: The Documentation Drift

Symptom: Docstrings and audit comments reference axiom counts, file names, or proof strategies that are no longer current.

Example: After the Great Audit, 14 files still referenced "45 axioms" (the pre-audit count) or cited archived files as active dependencies.

Fix: Treat docstrings as code. Include them in the audit cycle. Use grep-based validation to detect stale references.

7.5 Anti-Pattern 5: The Forward Reference

Symptom: A theorem is used before its definition in the same file, causing a Lean compilation error.

Example: `oct_gap_positive` (line 392) used `oct_gap_dominates_derived` (line 640) in `ClassRestriction.lean`.

Fix: Lean enforces top-to-bottom declaration order within a file. When refactoring, always check that moved declarations respect this ordering. Consider splitting large files (> 500 lines) into smaller modules.

8 The Testing Stack

8.1 Type Checking as Testing

In Lean 4, type checking *is* the test. If `lake build` succeeds with zero errors, every theorem is correct with respect to the stated axioms. There is no separate test suite—the type checker is the test suite.

However, type checking alone does not catch:

- **Unintended axioms**: A theorem may type-check while depending on more axioms than intended.
- **Vacuous truth**: A theorem with an unsatisfiable hypothesis is trivially true but useless.
- **Wrong statement**: The theorem may be provable but not say what the user intended.

8.2 Companion Numerical Validation

We use Rust programs as an independent validation layer:

- **Gram Oracle**: Computes Gram matrix entries, eigenvalues, and Rayleigh quotients using exact rational arithmetic (211,202 lines of Rust). Validates that the formal L^2 identity matches numerical computation to 15 digits.
- **Contour Oracle**: Computes the three-term frequency-domain decomposition of the Parseval Bridge using parallel numerical integration. Validates the algebraic identity $d_N^2 = T_1 - T_2 + T_3$ to machine precision.

The Rust programs cannot replace formal verification (they use floating-point arithmetic and cannot prove universally quantified statements), but they catch *wrong statements*: if a formally proved identity disagrees with numerical computation, either the axioms are wrong or the Rust code has a bug.

8.3 Compiler Warning Discipline

We enforce a strict warning discipline:

Warning Type	**Policy**
Unused variable	Prefix with _
Deprecated API	Replace if possible; document if not
No-op tactic	Remove
Unused `simp` argument	Omit
`#check`/`#print` output	Comment out before commit
`sorry` on critical path	Blocked (zero-sorry policy)
`sorry` on alternative path	Documented and tracked

As of the final build, there are zero actionable warnings, 2 intentional `sorry` markers (on an alternative proof path, not the crown path), and 4 cosmetic Lean linter messages.

9 Lessons Learned

1. **Archive everything.** The 54% discard rate means that more than half of all code written was eventually superseded. But archived code is not wasted—it documents the exploration space and often contains reusable lemmas.

2. **Axioms are technical debt.** Every axiom is a promise to prove something later. Like technical debt in software, axioms accumulate interest: they make the proof harder to audit, inflate reported metrics, and create ghost axiom risks. Pay them down aggressively.

3. **`#print axioms` is your profiler.** Run it on every top-level theorem after every refactoring. Unexpected axioms in the output are the proof-engineering equivalent of performance regressions.

4. **The lakefile is a manifest.** Treat it like a `package.json` or `Cargo.toml`: it must exactly reflect the filesystem. Stale entries cause build failures; missing entries cause silent omissions.

5. **Docstrings are code.** In a codebase that evolves rapidly, documentation drifts within days. Include docstring accuracy in the audit checklist.

6. **Numerical validation catches wrong statements.** A formally verified theorem can be correct but useless (wrong statement, vacuous hypothesis). Companion experiments in a different language provide an independent sanity check.

7. **Modularity enables parallelism.** The 11-module architecture allowed independent work on different proof fronts (spectral engine, sieve engine, Mellin bridge) without merge conflicts. The assembly layer at the top integrates all paths.

8. **The AI is the fast typist, not the strategist.** In our workflow, mathematical strategy was always human-driven. The AI's primary value was in Lean syntax, Mathlib API discovery, codebase navigation, and automated refactoring—tasks that are high-volume but low-creativity.

10 Related Work

Large-scale formalizations in proof assistants include:

- **Gonthier et al.** (2013): Four-Color Theorem in Coq. ∼60,000 LOC, zero axioms.
- **Hales et al.** (2017): Kepler Conjecture (Flyspeck) in HOL Light. ∼300,000 LOC over 12 years.
- **Scholze–Buzzard** (2022): Liquid Tensor Experiment in Lean 4. ∼60,000 LOC.
- **Tao et al.** (2023): Polynomial Freiman-Ruzsa conjecture in Lean 4. Community-driven, ∼25,000 LOC.

The Cathedral is distinguished by: (a) the high axiom count (40, reflecting the open status of RH), (b) the systematic axiom lifecycle management, (c) the high discard rate (54%), and (d) the compressed development timeline (23 days, enabled by human-AI pair programming).

11 Conclusion

The Cathedral demonstrates that large-scale formal verification of frontier mathematics is feasible with current tools (Lean 4, Mathlib) and accelerated by human-AI collaboration. The primary engineering challenges are not in the verification itself—Lean's type checker handles that—but in the *lifecycle management* of evolving proofs: keeping axioms synchronized with theorems, lakefiles synchronized with filesystems, and documentation synchronized with code.

The ghost axiom problem (Section 4) is, to our knowledge, the first documented instance of this failure mode in the proof engineering literature. We expect it to become more common as proof codebases grow beyond the scale that a single person can audit manually.

The tools are ready. The methodology is developing. The next cathedral—whatever it proves—will be built faster.

Source code: `https://github.com/jrgochan/prime`

References

[1] L. de Moura *et al.*, *The Lean 4 Theorem Prover and Programming Language*, CADE-28, 2021.

[2] The Mathlib Community, *Mathlib: A unified library of mathematics formalized in Lean 4*, `https://github.com/leanprover-community/mathlib4`.

[3] G. Gonthier, *Engineering Mathematics: The Odd Order Theorem Proof*, POPL 2013, 1–2.

[4] T. Hales *et al.*, *A Formal Proof of the Kepler Conjecture*, Forum of Mathematics, Pi **5** (2017).

[5] J. Commelin and P. Scholze, *Liquid Tensor Experiment*, `https://github.com/leanprover-community/liquid`, 2022.

[6] T. Tao *et al.*, *A Formalization of the Polynomial Freiman-Ruzsa Conjecture in Lean 4*, 2023.

[7] L. Báez-Duarte, *A strengthening of the Nyman–Beurling criterion*, Atti Accad. Naz. Lincei Rend. Mat. Appl. **14** (2003), 5–11.